

Fundamental Problems

COMPCSI 752: Big Data Management

University of Auckland

Credit to Ninh Pham for these slides

Outline

- Web crawling
 - Near-duplicate detection
- Web search
 - PageRank
- Data cleaning
 - tf-idf representation
 - Similarity join



INTERNET GIANTS THAT RULE THE WEB



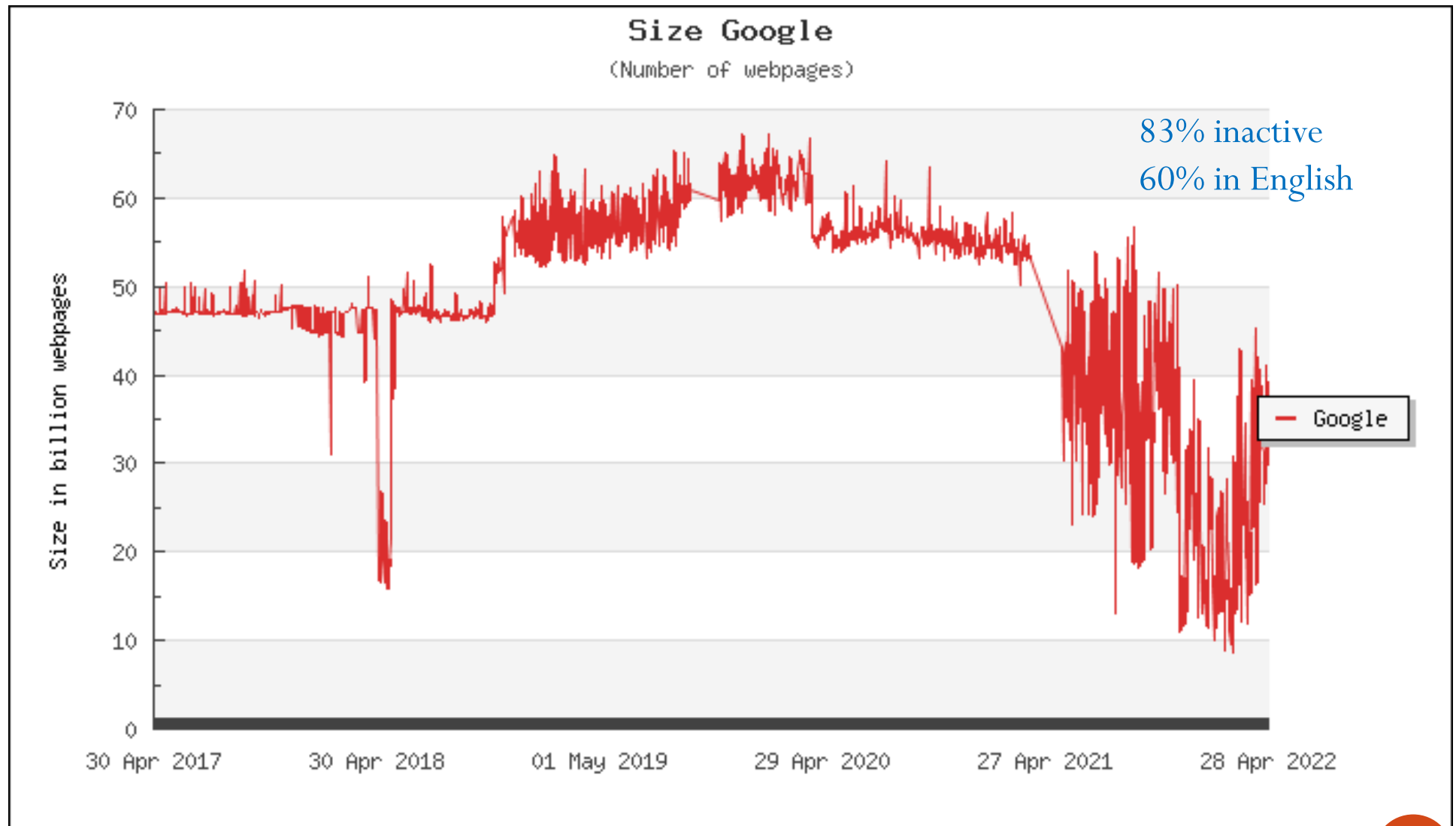
The top 20 multi-platform online properties over time



Sources: Media Metrix (Dec '98), ComScore (Dec '03, Dec '08, Dec '13, Nov '18, and Nov '21) *Universal Music Group **Meredith acquired by Dotdash in December 2021

VISUALCAPITALIST.COM

Internet size



Surface Web

Exploring The Hidden Internet

4%

Deep Web

90%

Legal Documents

Government Records

Scientific Reports

Academic Records

Financial Records

Dark Web

6%

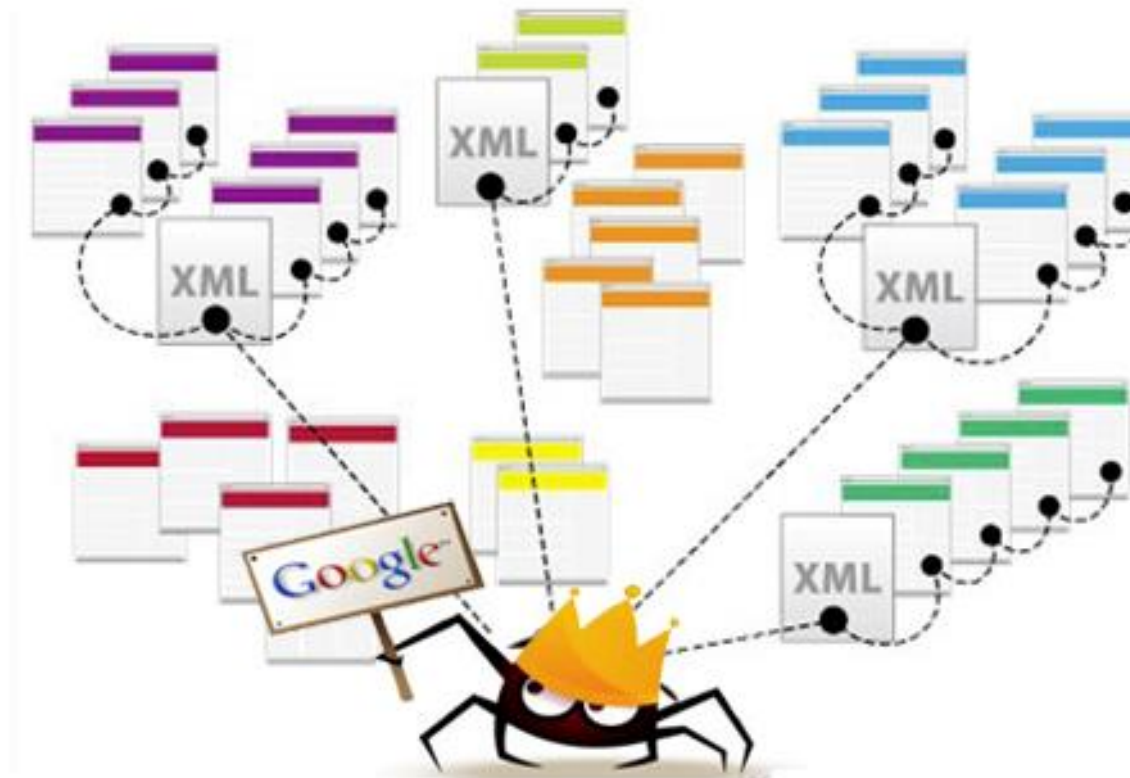
Drugs

Tor

Illegal Info

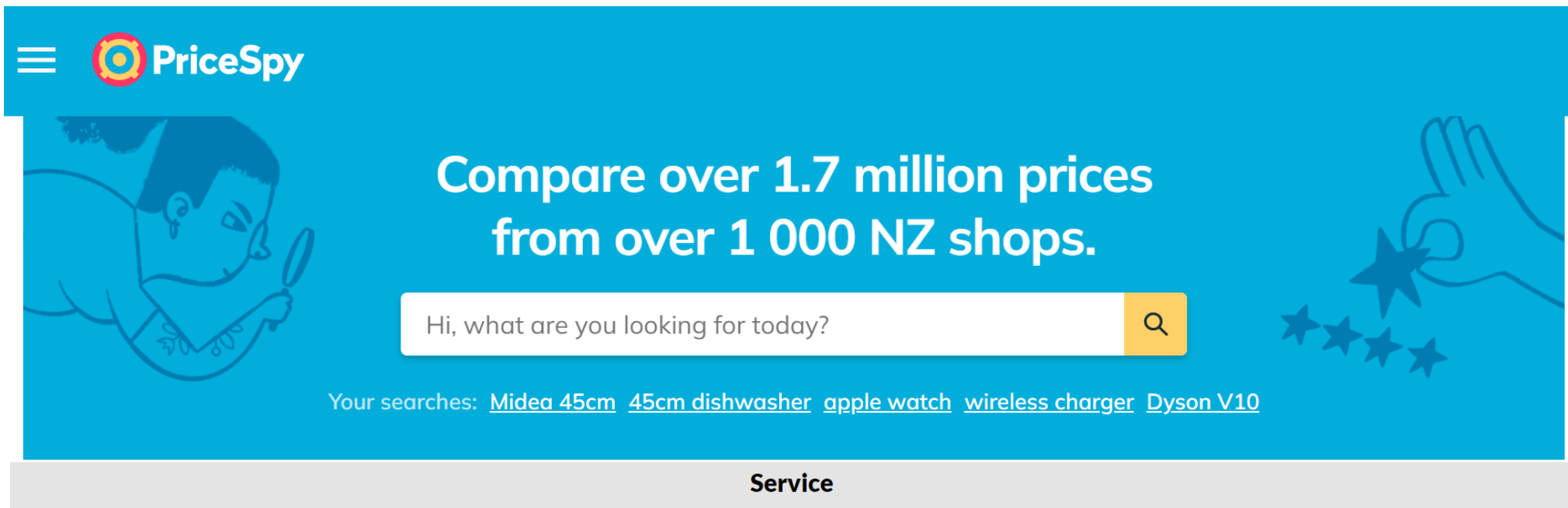
Web crawlers

- Crawlers (spiders, robots): autonomous user agents that retrieve pages from the Web



Why we need web crawlers?

- Supplier pricing and availability



The image shows a banner for the PriceSpy website. On the left, there is a blue background with a white icon of a person looking through a magnifying glass. To the right of this, the text "Compare over 1.7 million prices from over 1 000 NZ shops." is displayed in white. Below this text is a white search bar with the placeholder text "Hi, what are you looking for today?" and a yellow search button with a magnifying glass icon. To the right of the search bar, there is a blue background with a white icon of a hand holding a star. Below the search bar, the text "Your searches: Midea 45cm 45cm dishwasher apple watch wireless charger Dyson V10" is displayed in white. The PriceSpy logo is in the top left corner. A grey bar with the word "Service" is at the bottom of the banner.

PriceSpy

Compare over 1.7 million prices from over 1 000 NZ shops.

Hi, what are you looking for today?

Your searches: [Midea 45cm](#) [45cm dishwasher](#) [apple watch](#) [wireless charger](#) [Dyson V10](#)

Service

Why we need web crawlers?

- Monitor news and social media for **sentiment analysis**
 - Improve contact centers and customer services in Vodafone



What is sentiment analysis (and is it worth the investment)?

<https://www.vodafone.com/business/customer-experience/what-is-sentiment-analysis-and-is-it-worth-the-investment>

Web crawlers

- No real termination condition (virtual unlimited number of Web pages!)
- Graph-browsing problem:
 - Depth-first: not very adapted, possibility of being lost in robot traps
 - Breadth-first
 - Combination of both: breadth-first with limited depth-first on each discovered website

Sources of new URLs

- From HTML pages:
 - hyperlinks `<a href="...">...</a>`
 - media `` `<embed src="...">` `<object data="...">`
 - frames `<frame src="...">` `<iframe src="...">`
 - JavaScript links `window.open("...")`
- Other hyperlinked content (e.g., PDF files)
- Non-hyperlinked URLs that appear anywhere on the Web (in HTML text, text files...): use regular expressions to extract

Identification of duplicate Web pages

- Problem:

- Identifying duplicates or near-duplicates on the Web to prevent multiple indexing and processing

- Trivial duplicates: same resource at the same **canonized** URL:

- <http://wikipedia.com>
- <http://www.wikipedia.com/?source=asdf>

- Exact duplicates: identification by **hashing**

- Near-duplicates: (big data challenge)

Hashing

- Definition:

- A hash function is a deterministic ^{确定性} mathematical function transforming objects (numbers, strings, binary, ...) into **fixed-size** random integers

- Example:

- Java hash function for the *String* class:

$$\sum_{i=0}^{n-1} s_i \times 31^{n-i-1} \bmod 2^{32}$$

- where s_i is the (Unicode) code of character i of a string s

Exact duplicates: Example

- Given three webpages with the contents as follows
 - S1 = “Coronavirus”
 - S2 = “Coronavirus”
 - S3 = “Covid-19-Coronavirus”
- Applying hash function $h: [\text{string}] \rightarrow [2^{32}]$ over the content
 - $h(S1) = 12345$
 - $h(S2) = 12345$
 - $h(S3) = 13$
 - Hence S1 and S2 are duplicates
 - The probability $\Pr [h(S3) = h(S1)]$ is extremely small (e.g. $1/2^{32}$)

极小概率

Near-duplicates: Example

- Given three webpages with the contents as follows
 - S1 = “Coronavirus”
 - S2 = “Corona-virus”
 - S3 = “Covid-19-Coronavirus”
- Define a similarity / distance measure between these strings
 - Edit distance
 - Jaccard similarity
 - Containment similarity
 - ...

Near-duplicate detection

- Edit distance:

最少修改次数.

- Count the minimum number of basic modifications (additions, deletions or substitution of characters or words, etc.) to obtain a document from another one
- Example:
 - $ED(\text{"Coronavirus"}, \text{"Corona-virus"}) = 1$
 - $ED(\text{"Coronavirus"}, \text{"Covid-19-Coronavirus"}) = 9$
- Good measure of similarity, with $O(mn)$ complexity where m and n are the size of the strings/documents
- But: **does not scale** to a large collection of documents (unreasonable to compute the edit distance for every pair!)

Near-duplicate detection

- Shingles:

相同 k -gram 数

- Idea: two documents similar if they mostly share the **same succession of k -grams** (succession of tokens of length k)
- Example with $k = 2$:
 - **D1** = “i like to watch the sun set with my friend”
 - **D2** = “my friend and I like to watch the sun set”

Near-duplicate detection

- Shingles:

- Idea: two documents similar if they mostly share the **same succession of k -grams** (succession of tokens of length k)
- Example with $k = 2$:
 - **D1** = “i like to watch the sun set with my friend”
 - **S** = {i like, like to, my friend, set with, sun set, the sun, to watch, watch the, with my}
 - **D2** = “my friend and I like to watch the sun set”
 - **T** = {my friend, friend and, and I, I like, like to, to watch, watch the, the sun, sun set}

Near-duplicate detection

- Shingles:

- Idea: Two documents similar if they mostly share the **same succession of k -grams** (succession of tokens of length k)

$k \uparrow$

accurate. $\uparrow$

更严格.

- Example with $k = 2$:

- **D1** = “i like to watch the sun set with my friend”
- **S** = {i like, like to, my friend, set with, sun set, the sun, to watch, watch the, with my}
- **D2** = “my friend and I like to watch the sun set”
- **T** = {my friend, friend and, and I, I like, like to, to watch, watch the, the sun, sun set}

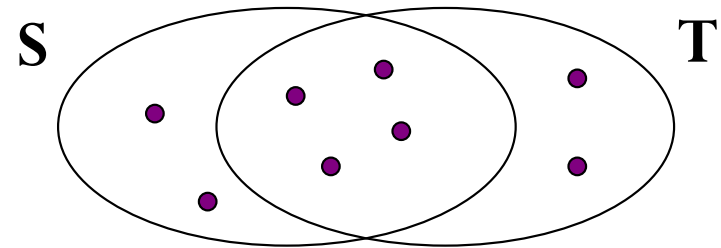
Jaccard similarity

- Jaccard similarity of the 2 sets of shingles **S** and **T**:

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|} = \frac{\text{intersection size}}{\text{union size}}$$

- Example:

- $J(S, T) = 4/8 = 1/2$



- **D1** = “I like to watch the sun set with my friend”
 - **D2** = “My friend and I like to watch the sun set”
 - Using 2-shingles, $J(D1, D2) = 7/11$

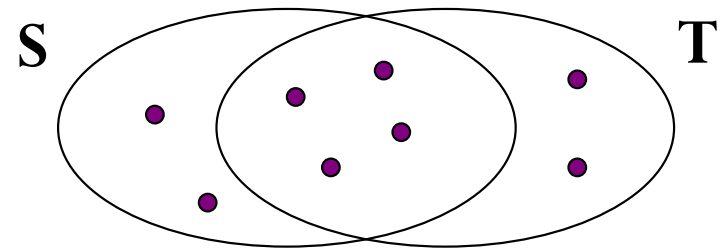
Containment similarity

- Containment similarity of **S** in **T**:

$$C(S, T) = \frac{|S \cap T|}{|T|} = \frac{\text{intersection size}}{\text{size of T}}$$

- Example:

- $C(S, T) = 4/6 = 2/3$

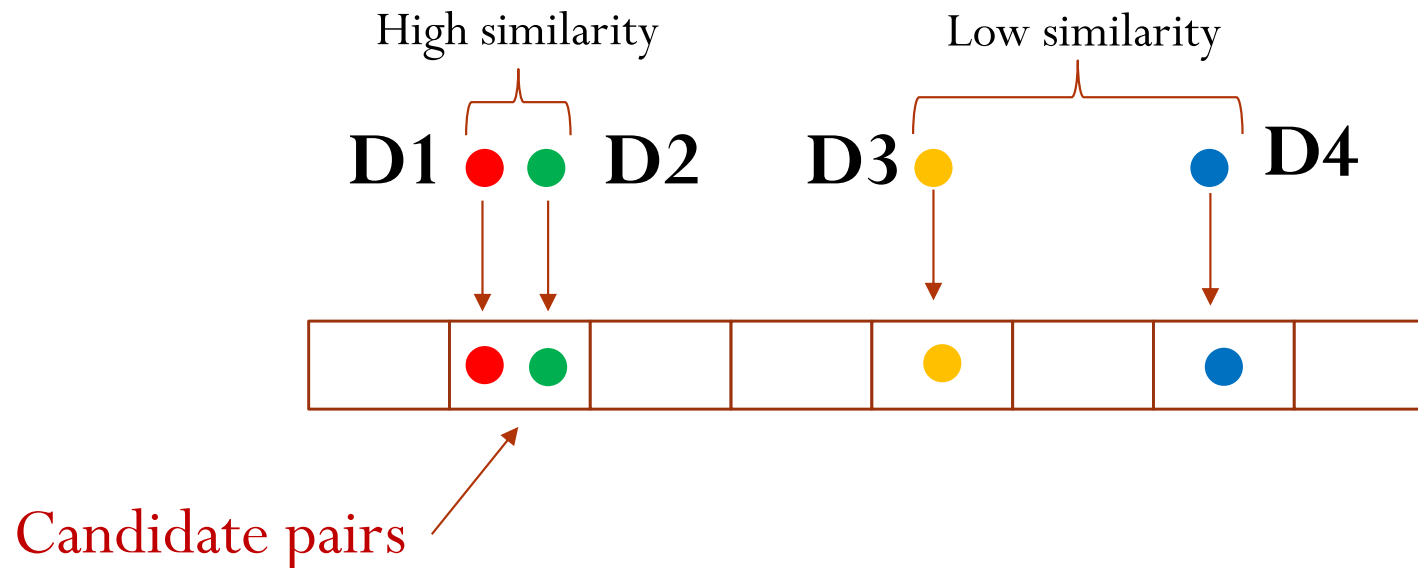


- D1** = “I like to watch the sun set with my friend”
- D2** = “My friend and I like to watch the sun set”
- Using 2-shingles, C(D1, D2) = 7/9

看前四,

Near-duplicate detection

- Scalability: 可扩展性.
 - Time: All pair similarity using Jaccard/containment similarity
 - Space: Large space usage for representing the sets of shingles
- Locality-sensitive hashing (minwise hashing) to save time:



Near-duplicate detection

- Scalability:
 - Time: All pair similarity using Jaccard/containment similarity
 - Space: Large space usage for representing the sets of shingles
- Odd sketch to save space:
 - Idea: Representing the sets of shingles as fixed size bit vectors
 - Exploit extremely fast binary operations, e.g. XOR, `_builtin_popcount` of compilers

Near-duplicate detection application: Matching with profiles

- **Problem:** Find a matching pair such that their profiles are different **at most** one?

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Matching with profiles

- **Problem:** Find a matching pair such that their profiles are different **at most** one?

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Matching with profiles

- **Problem:** Find a matching pair such that their profiles are different **at most** one?

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: HCM



Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: CPH



Age: 25 – 30

Like: Music

Hate: Sport

Job: Singer

City: NY



Odd Sketch: Matching with profiles

0	0	0	0
---	---	---	---

0	0	0	0
---	---	---	---

0	0	0	0
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

0	0	0	0
---	---	---	---

0	0	0	0
---	---	---	---

0	0	0	0
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30

Like: Sport
Hate: Music
Job: Teacher
City: HCM



Age: 25 – 30

Like: Sport
Hate: Music
Job: Teacher
City: CPH



Age: 25 – 30

Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

hash

0	0	0	0
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: HCM



0	0	0	0
---	---	---	---

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: CPH



0	0	0	0
---	---	---	---

Age: 25 – 30

Like: Music

Hate: Sport

Job: Singer

City: NY



Odd Sketch: Matching with profiles

flip

0	1	0	0
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: HCM



0	1	0	0
---	---	---	---

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: CPH



0	1	0	0
---	---	---	---

Age: 25 – 30

Like: Music

Hate: Sport

Job: Singer

City: NY



Odd Sketch: Matching with profiles

hash

0	1	0	0
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: HCM



0	1	0	0
---	---	---	---

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: CPH



0	1	0	0
---	---	---	---

Age: 25 – 30

Like: Music

Hate: Sport

Job: Singer

City: NY



Odd Sketch: Matching with profiles

flip

0	1	0	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: HCM



0	1	0	1
---	---	---	---

Age: 25 – 30

Like: Sport

Hate: Music

Job: Teacher

City: CPH



0	1	1	0
---	---	---	---

Age: 25 – 30

Like: Music

Hate: Sport

Job: Singer

City: NY



Odd Sketch: Matching with profiles

hash

0	1	0	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



0	1	0	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	1	0
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

翻转
flip

0	0	0	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



0	0	0	1
---	---	---	---

flip back

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	0	0
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

hash

0	0	0	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



0	0	0	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	0	0
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

flip

$h: str \rightarrow [4]$

1	0	0	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



1	0	0	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	1	0
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

hash

1	0	0	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



1	0	0	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	1	0
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

flip

1	0	1	1
---	---	---	---

$h: str \rightarrow [4]$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



1	0	0	0
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



0	1	1	1
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles

 x

1	0	1	1
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM

 q

1	0	0	0
---	---	---	---

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH

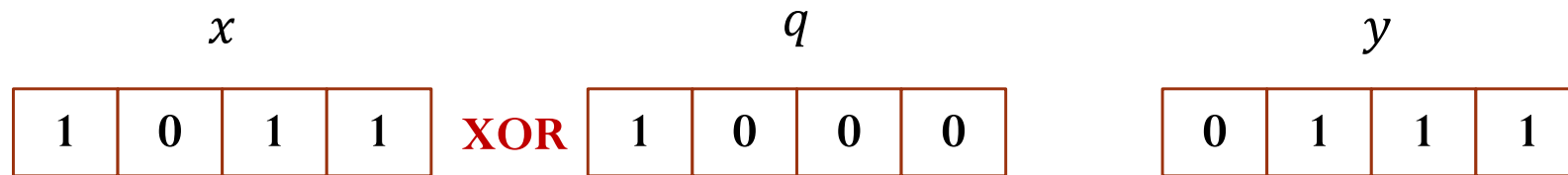
 y

0	1	1	1
---	---	---	---

Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles



$$|x \Delta q| = 2$$

Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



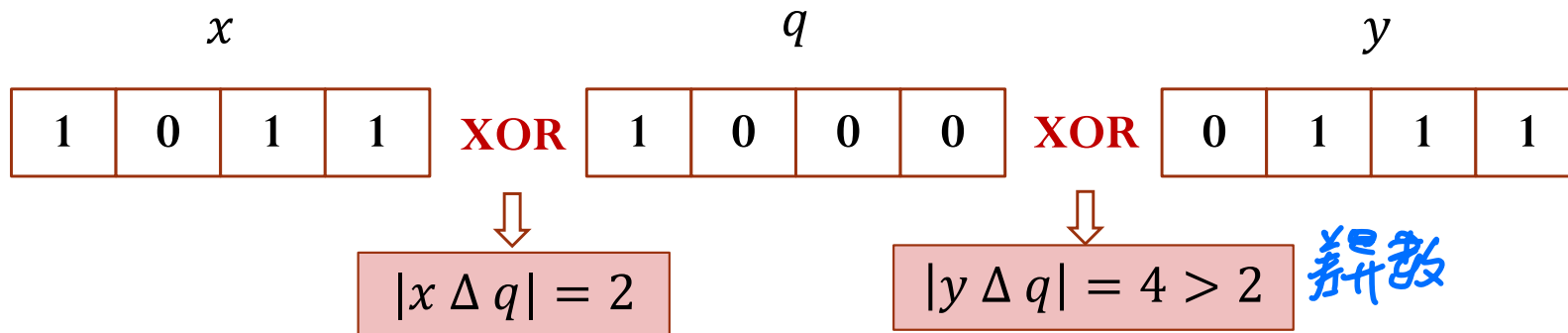
Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH



Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketch: Matching with profiles



Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: HCM



Age: 25 – 30
Like: Sport
Hate: Music
Job: Teacher
City: CPH

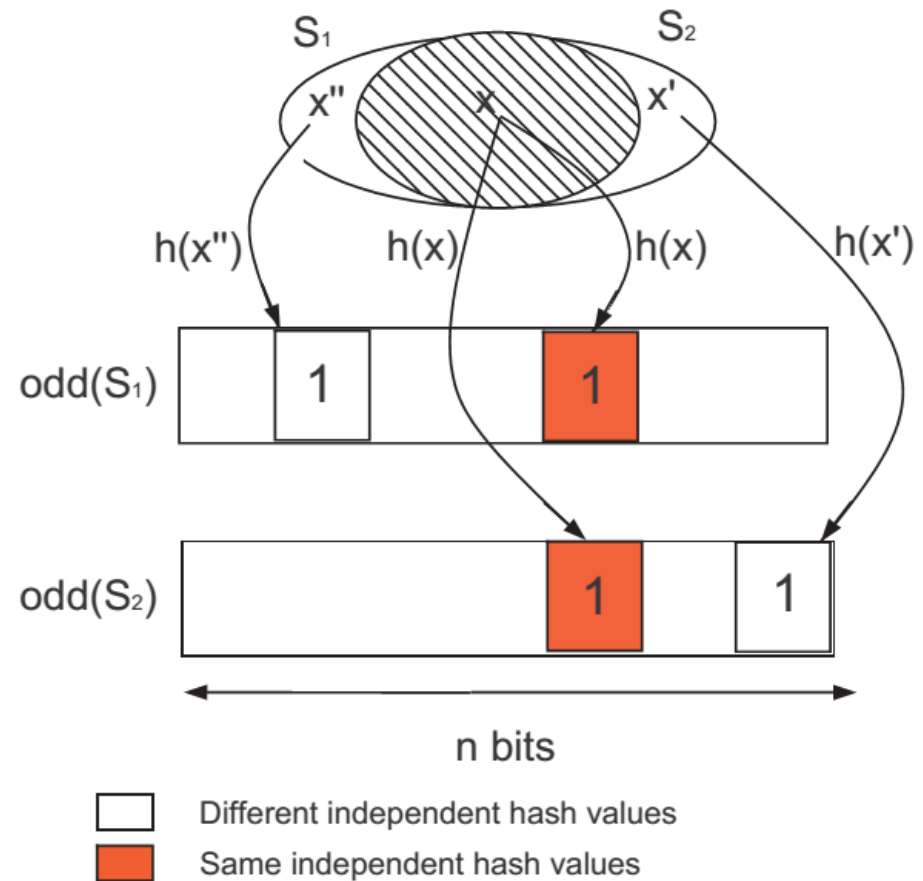


Age: 25 – 30
Like: Music
Hate: Sport
Job: Singer
City: NY



Odd Sketches

- Efficient estimation for sets of high similarity
- Published in WWW'14



- Build $\text{odd}(S)$ of m items is identical to randomly throw m balls into n bins, and store the parity for each bin
- $\text{odd}(S_1) \oplus \text{odd}(S_2) = \text{odd}(S_1 \Delta S_2)$

Homework

- Implement the odd sketch with Python to solve the near-duplicate detection
 - Use the [KOS blog entries](https://archive.ics.uci.edu/ml/datasets/Bag+of+Words) data set from <https://archive.ics.uci.edu/ml/datasets/Bag+of+Words>
 - This data set consists of **3430** documents, each represented as a set of keywords. The total number of keywords of the corpus is **6960**
 - Finding all matching pair of documents whose keywords are different at most **10**, e.g. the set difference size between two documents is at most **10**
 - [Hint](#): odd sketch with **40** bits is enough